Agda by Example: Programming and Proving with Dependent Types

Peter Thiemann

Freiburg University

March 13, 2026

Motivation

- We want software that is **correct by construction**, not "probably ok after tests".
- Types as **specifications**: function signatures as contracts.
- Dependent types let types talk about **values**: lengths, bounds, invariants.
- Agda fulfills two roles:
  - a **total functional programming language**, and
  - an **interactive proof assistant**.

Agda in a Nutshell

- Agda is a proof assistant and dependently typed FP language based on **intuitionistic Martin–Löf type theory** (constructive proofs-as-programs).
- Agda is named after the Swedish song "Hönan Agda" about a hen, serving as a pun on the proof assistant Coq (another dependently typed proof assistant of French origin).
- Agda was primarily developed by **Ulf Norell** (notably in his 2007 PhD work), taking inspiration from ALF, then grown by a broader community into today's widely used system.
- Implemented in **Haskell**; emphasizes **interactive development** (holes), and **dependent pattern matching**.
- The **Agda standard library** provides reusable foundations (data structures, algebra, equality reasoning) and strongly shapes idiomatic Agda.
- Good for **mechanized metatheory**, **certified programming**, and teaching the tight link between **types, specs, and proofs**.

What We Will Do (90 min)

1. Warm-up: simple programs, total functions.
2. Inductive data types: lists, recursion.
3. Dependent types in action: vectors indexed by length.
4. Propositions as types: equality and proofs.
5. Case study: a small verified function.

Goal: Leave with a mental model for using types as specifications, and how to explore that space using Agda.

Agda as a Language + Proof Assistant

- Source files: `.agda` or literate `.lagda.md`.
- Interactive editing:
  - Emacs, VSCode, etc.
  - Holes: typed as ?, typeset as { }; most commands fill/refine holes.
- Everything is total:
  - Functions are defined by exhaustive pattern matching.
  - Termination is checked.

Warm-Up: Simple Functions

```
module Warmup where

open import Agda.Builtin.Nat

double : Nat → Nat
double n = n + n

triple : Nat → Nat
triple n = {!!}
```

Exercise 1 (few minutes):

- Fill the hole for `triple`.
- Check the types with `C-c C-,`.
- Refine the hole with `C-c C-r`.
- Experiment with `C-c C-n` to normalize expressions.

Key idea: Types as a starting point; holes as "typed to-do items".

Lists from Scratch

```
data List (A : Set) : Set where
  []   : List A
  _::_ : A → List A → List A

length : {A : Set} → List A → Nat
length []       = 0
length (x :: xs) = 1 + length xs
```

Key points:

- Inductive data = constructors + recursion.
- Types track element type (A), not size (yet).
- Pattern matching drives computation.

Appending Lists

```
append : {A : Set} → List A → List A → List A
append []       ys = ys
append (x :: xs) ys = x :: append xs ys
```

Exercise 2:

- Implement `reverse : {A : Set} -> List A -> List A`.
- Use pattern matching and recursion; load the file and let Agda check totality and termination.

Observation:

- Types specify *shape* of data, but not *properties* like length.

Vectors: Lists with Length in the Type

```
open import Agda.Builtin.Nat

data Vec (A : Set) : Nat → Set where
  []   : Vec A 0
  _::_ : {n : Nat} →
         A → Vec A n → Vec A (suc n)
```

Idea:

- Vec A n is a list of A *indexed* by length n.
- Indices (here: Nat) are values appearing in types.
- Invariants like "length is preserved" become type-level facts.

Mapping over Vectors

```
vmap :
  {A B : Set} {n : Nat} →
  (A → B) → Vec A n → Vec B n
vmap f []        = []
vmap f (x :: xs) = f x :: vmap f xs
```

Reading the type:

- For any length n, if you give me a vector of A of length n,
- I return a vector of B of the *same* length n.

Exercise 3:

- Define vappend : Vec A m -> Vec A n -> Vec A (m + n).

Equality as a Type

In Agda, propositional equality is a data type:

```
data _≡_ {A : Set} (x : A) : A → Set where
  refl : x ≡ x
```

- A value of type x ≡ y is evidence that x and y are equal.
- There is only one constructor: `refl`, proving x ≡ x.
- To prove x ≡ y, we typically normalize both sides and use `refl`.

A Simple Proof: Right Identity of +

```
open import Agda.Builtin.Nat
open import Agda.Builtin.Equality

zero-right-id : (n : Nat) → n + 0 ≡ n
zero-right-id zero    = refl
zero-right-id (suc n) = cong suc (zero-right-id n)
```

Notes:

- cong lifts equality through a function:

```
cong : {A B : Set} (f : A → B) {x y : A} →
       x ≡ y → f x ≡ f y
```

- The proof is just a recursive function following the structure of `Nat`.
- It constitutes a proof by mathematical induction that $\forall n. n + 0 = n$.

List Property: Appending []

```
length-append-[] :
  {A : Set} (xs : List A) →
  length (append xs []) ≡ length xs
length-append-[] [] = ?
length-append-[] (x :: xs) = ?
```

Exercise 4:

- Type this lemma and ask Agda for goals (holes).
- Fill in the recursive case using cong.

Takeaway:

- Proofs look like ordinary recursive functions.
- The type encodes the property.

List Property: Left and Right Id

```
empty-right-id :
  {A : Set} (xs : List A) →
  append xs [] ≡ xs
empty-right-id = ?

empty-left-id :
  {A : Set} (xs : List A) →
  append [] xs ≡ xs
empty-left-id = ?
```

Exercise 4a:

- Type the empty-right lemma and ask Agda for goals (holes).
- Fill in the recursive case.

Exercise 4b:

- Type the empty-left lemma and ask Agda for goals (holes).
- Do you need induction?

List Property: Left and Right Id

```
empty-right-id :
  {A : Set} (xs : List A) →
  append xs [] ≡ xs
empty-right-id = ?

empty-left-id :
  {A : Set} (xs : List A) →
  append [] xs ≡ xs
empty-left-id = ?
```

Exercise 4a:

- Type the empty-right lemma and ask Agda for goals (holes).
- Fill in the recursive case.

Exercise 4b:

- Type the empty-left lemma and ask Agda for goals (holes).
- Do you need induction?

Takeaway:

- Some properties hold *definitionally*, i.e., by normalizing according to function definitions.

Safe Indexing with Fin

```
data Fin : Nat → Set where
  fzero : {n : Nat} → Fin (suc n)
  fsuc  : {n : Nat} → Fin n → Fin (suc n)
```

Idea:

- `Fin n` represents numbers `0 .. n-1`.
- This range corresponds to the domain of `Vec A n`!
- Combining `Fin n` with `Vec A n` yields safe lookup.
- It is impossible to construct an out-of-bounds index.

Lookup That Cannot Fail

```
lookup :
  {A : Set} {n : Nat} →
  Fin n → Vec A n → A
lookup fzero    (x :: xs) = x
lookup (fsuc i) (x :: xs) = lookup i xs
```

Key observations:

- No case for `[]` is needed: `Fin 0` is empty (recognized by Agda).
- No runtime bounds checking.
- The type guarantees safety by construction.

Exercise 5:

- Implement a small function using `lookup` (e.g., get the head of a non-empty vector).

Challenge Problem

```
vsnoc : {A : Set} {n : Nat} → Vec A n → A → Vec A (suc n)
vsnoc xs x = {!!}

vreverse : {A : Set} {n : Nat} → Vec A n → Vec A n
vreverse [] = []
vreverse (x :: xs) = vsnoc (vreverse xs) x
```

Exercise 6 (challenge):

- Try to implement a function vsnoc to attach a single element to the *end* of a vector. This function can be use to implement `vreverse` to reverse a vector.
- Analyze the problem you run into

What We Saw

- **Inductive data** for structure (Nat, List, Vec).
- **Dependent types** to encode invariants (length, bounds).
- **Equality as a type** and small composable proofs.
- **Safe APIs** by design:
  - `Vec A n`, `Fin n`, `lookup`.

Takeaways for Day-to-Day Work

After this tutorial, you should be able to:

- Encode key invariants in types to design safer APIs.
- Formulate precise specifications as type signatures.
- Apply proof-driven reasoning to structure and validate behavior.

Where to Go Next

- Agda standard library documentation.
- "Programming Language Foundations in Agda" (PLFA).
- Explore:
  - More advanced dependently typed programming patterns.
  - Larger verification case studies: interpreters, DSLs, protocols.

Thank you!